NoiseCore VM

User's Manual

Version 1.2.0

A substrate-mediated virtual computation environment grounded in the Extended Binary Golay Code [24,12,8]

E R A Craig / UBP Research — May 2026

208 tests passing • 46 opcodes • 8 example programs

What's in this manual
Complete, self-contained reference — no dependency on earlier versions
46-instruction ISA with full operand syntax and flag behaviour
Full assembly language (.nca) syntax, directives, and conventions
Python API: run_program, CPU, Assembler, Device, SubstrateALU, ShadowRegister
Substrate mode: Golay geometry, Shadow Processor, S flag, carry work, NRCI
SubstrateALU: base-12 native arithmetic (ADD/SUB/MUL/DIV on digit cells)
CLI reference, 8 worked examples, debugging guide, known limitations

1. Introduction

NoiseCore VM is a fully functional virtual CPU whose registers and memory are backed by the Golay [24,12,8] geometric substrate of the Universal Binary Principle (UBP) framework. It is not a simulation of a CPU: it is a real register machine with a 46-instruction ISA, a two-pass symbolic assembler, a disassembler, a trace recorder, and eight working example programs.

The system serves two purposes simultaneously.

- As a computational tool: programs written in NoiseCore Assembly (.nca) can perform any task a general-purpose integer processor can — arithmetic, recursion, sorting, string manipulation, prime sieves. The CPU runs at over 600,000 instructions per second in plain mode.
- As a UBP research instrument: switching to substrate mode replaces each register with a ShadowRegister backed by a real Golay codeword manifold. As of v1.2.0, all arithmetic (ADD/SUB/MUL/DIV) is performed natively by the SubstrateALU — digit-by-digit on the base-12 cell arrays. Every stored value leaves a measurable geometric fingerprint. The S (Shadow) flag reveals when computation crosses the Voronoi boundary of the linear Phenomenal regime.

1.1 System at a Glance

Component	What it provides
CPU	8 general-purpose registers (R0–R7), 256-cell memory, 5-flag register {Z N C V S}, PC, cycle counter, trace recorder
ISA	46 instructions across 6 groups: arithmetic/logic, control flow, immediate-mode, memory, stack/subroutines, syscalls/misc
Assembler	Two-pass .nca source files with labels, .equ constants, .data sections, char literals, hex/binary immediates
Disassembler	Bytecode → labelled assembly text; round-trip verified
SubstrateALU (v1.2.0)	Base-12 digit-chain arithmetic: ADD/SUB via carry/borrow, MUL via long-mul, DIV/MOD via Knuth Algorithm D. Python arithmetic operators never applied to full register values.
Substrate mode	ShadowRegisters: Golay-backed with shadow_active detection, carry_work accounting, NRCI and magnitude SHA fingerprints
CLI	noisecore run / asm / disasm / repl / info
Tests	208 passing tests across 12 test modules

1.2 What This Manual Covers

- Installation and the two-minute quick-start
- The architecture: layers from Python API down to the Golay substrate
- The assembly language (.nca): full syntax, directives, operand forms
- Complete instruction reference with worked examples
- The Python API for embedding the VM in research scripts
- Substrate mode: Golay geometry, Shadow Processor, S flag, carry work, NRCI, magnitude SHA
- SubstrateALU: native base-12 arithmetic for all CPU operations
- The command-line interface
- Practical application patterns: algorithms, I/O, subroutines, research patterns
- Debugging and troubleshooting
- Known limitations and design trade-offs

2. Installation

2.1 Requirements

- Python 3.10 or later
- The UBP golay_engine on the Python path — provides real [24,12,8] Golay code machinery.

Without it, a geometrically accurate stub preserving the displacement curve is used automatically.

2.2 Install from Source (editable)

```
cd noisecore_vm_v1_2_0/
pip install -e .
```

2.3 Verify Installation

```
noisecore info        # prints the full 46-opcode ISA table
pytest                # 208 tests, all green
```

2.4 Project Structure

```
noisecore_vm_v1_2_0/
  noisecore_vm/
    core/
      golay_engine.py      # [24,12,8] Golay code implementation
      substrate.py         # NoiseCellV3, NoiseRegisterV3, ShadowRegister,
                           # SubstrateALU, count_base4_carries
    vm/
      isa.py               # OpSpec definitions, 46 opcodes
      assembler.py         # two-pass assembler
      disassembler.py
      cpu.py               # CPU class, _op_* implementations
    __init__.py            # public API surface
  examples/                # 8 .nca programs
  tests/                   # 12 test modules, 208 tests
  docs/
    ARCHITECTURE.md
  CHANGELOG.md
```

3. Quick Start

3.1 Hello World in 30 Seconds

```
; hello.nca
LOAD    R0, 'H'
SYSCALL 2           ; PRINT_CHAR
LOAD    R0, 'i'
SYSCALL 2
SYSCALL 6           ; PRINT_NL
HALT

$ noisecore run hello.nca
Hi
```

3.2 Python API — Four Lines

```
from noisecore_vm import run_program

cpu, info = run_program('''
```

```
    LOAD  R0, #5
    LOAD  R1, #7
    ADD   R2, R0, R1
    MOV   R0, R2
    SYSCALL 1     ; PRINT_INT
    SYSCALL 6     ; PRINT_NL
    HALT
''')
print(cpu.console.output())   # -> 12
print(info['cycles'])         # -> 7
```

3.3 Shadow Processor Demo

When substrate_mode=True, the S flag latches whenever a register digit exceeds 4 — the Golay elastic limit.

```
from noisecore_vm import CPU, assemble

cpu = CPU(substrate_mode=True)
prog = assemble('LOAD R0, #5  \n  HALT')
info = cpu.run(prog.instructions)

print(info['shadow_flag'])        # 1 — S flag latched
print(info['total_carry_work'])   # cumulative base-4 carry events
fp = cpu.substrate_fingerprint()
print(fp['shadow_registers'])     # [0] — R0 is in Shadow Regime
```

digit 5 in base-12 → the cell crosses the Voronoi boundary of the Golay elastic limit (digits 0–4) → S latches to 1. See Chapter 8 for the full explanation.

3.4 SubstrateALU Demo — Native Base-12 Arithmetic

In substrate_mode=True, all arithmetic is performed digit-by-digit on the base-12 cell arrays. Python never operates on the full integer values.

```
from noisecore_vm import CPU, assemble

cpu = CPU(substrate_mode=True)
prog = assemble('''
    LOAD  R0, #1836
    LOAD  R1, #1839
    MUL   R2, R0, R1   ; base-12 long multiplication
    DIV   R3, R2, R1   ; Knuth Algorithm D (base-12)
    HALT
''')
info = cpu.run(prog.instructions)

print(cpu._read_reg(3))           # 1836 — 1836×1839÷1839
print(info['shadow_flag'])        # 1 — large digits entered Shadow Regime
print(info['total_carry_work'])   # base-12 carry events from the ALU
fp = cpu.substrate_fingerprint()
print(fp['shadow_registers'])     # registers in Shadow Regime
```

The MUL above is computed entirely by SubstrateALU — single-digit × single-digit products (≤121) accumulated through the base-12 cell array. Python never sees the integer 1836×1839. See Chapter 9 for the full algebra.

4. Architecture

4.1 The Five Layers

The system is cleanly separated into five layers. Each layer has a single responsibility and a documented interface to the layer below it.

```
┌──────────────────────────────────────────────────────────┐
│  User .nca source (labels, .equ, .data, char literals)   │
└──────────────────────┤ Assembler: two-pass ├──────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────┐
│  Bytecode: list of (opcode, *operands) tuples            │
└──────────────────────┤ CPU.execute() ├────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────┐
│  CPU: registers, memory, flags {Z N C V S}, PC, stack    │
└──────────────────────┤ _sau_add/sub/mul/div ├─────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────┐
│  SubstrateALU: base-12 digit-chain arithmetic            │
│  ADD/SUB: carry/borrow  MUL: long-mul  DIV: Knuth D      │
└──────────────────────┤ write_from_alu() ├─────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────┐
│  ShadowRegister → NoiseCellV3 → GolayCodeEngine          │
│  (base-12 positional accumulator on Golay [24,12,8])     │
└──────────────────────────────────────────────────────────┘
```

4.2 CPU Registers

The CPU has eight general-purpose registers, R0 through R7. R7 is the stack pointer (SP) by convention, initialised to memory_size−1 (address 255 by default). All eight registers can be used for computation; R7 is reserved by convention only, not enforced by hardware.

In substrate_mode=True, each register is a ShadowRegister — a NoiseRegisterV3 extended with Shadow Processor monitoring and a write_from_alu() entry point for native digit-chain writes from SubstrateALU. In substrate_mode=False, each register is a lightweight Python integer container that passes all the same ISA tests.

4.3 Sign-Magnitude Representation

The Golay substrate measures geometric displacement, which is non-negative. Signed integers are handled by sign-magnitude: the substrate register stores |value| (≥0), while a parallel boolean array in the CPU layer stores the sign bit.

```
# Internally:
cpu.registers[i].read()   ->  abs(value)   # substrate sees magnitude only
cpu._signs[i]             ->  True/False    # sign stored separately
cpu._read_reg(i)          ->  signed int    # reconstructed for ISA use
```

Negative numbers are fully supported at the ISA level (LOAD R0, #-7 works; SUB producing a negative result propagates correctly through SubstrateALU). The Golay substrate and SubstrateALU work exclusively with non-negative magnitudes; sign is resolved at the CPU dispatch layer.

4.4 Memory Model

- 256 cells, addresses 0–255 (configurable via memory_size parameter)
- .data directives populate cells starting from address 0, or from .org
- Stack lives at the top of memory (R7 = SP, starts at 255) and grows downward
- All memory accesses are bounds-checked; out-of-range raises IndexError
- In substrate mode, memory cells are NoiseRegisterV3 — shadow monitoring is on CPU registers only, not memory cells

4.5 Call Stack Design

Return addresses for CALL/RET are stored in a Python-level list (cpu.call_stack) that is separate from the data stack (R7/SP). This is a deliberate architectural choice:

- A data-stack imbalance cannot corrupt control flow
- Matches CPUs that maintain a separate link register (ARM) or hardware call stack (Cortex-M)
- cpu.call_stack is directly inspectable during debugging
- Recursive programs work correctly because the data stack and call stack are independent (see examples/03_factorial_recursive.nca)

4.6 Flag Register

The flag register contains five flags:

Flag	Set when	Cleared when
Z (Zero)	result == 0	result != 0
N (Negative)	result < 0	result >= 0
C (Carry/Borrow)	ADD: unsigned sum exceeds word_max / SUB: a < b (borrow)	otherwise
V (Overflow)	Signed result wraps outside [−2^(w−1), 2^(w−1)−1]	otherwise
S (Shadow)	Any register write causes a base-12 digit > 4 (Shadow Regime entry). Latching — stays 1 until CLRS or reset(). Always 0 in substrate_mode=False.	CLRS (0x52) instruction, or cpu.reset()

STORE, STOREI, STOREX, CALL, RET, JMP and all conditional jumps do not modify Z/N/C/V. CLRS clears only S.

4.7 Two Execution Modes

Mode	Speed	When to use

substrate_mode=False (default)	100k–1M+ ips	Unit tests, development, benchmarks, any work where you need only computed values
substrate_mode=True	~10–20× slower	When you need the S flag, shadow_active, carry_work, NRCI fingerprints, magnitude SHA, or SubstrateALU geometry

Both modes produce identical computational results. All 208 tests pass in both modes.

5. The Assembly Language (.nca)

5.1 Source File Format

A .nca file contains instructions, directives, labels, and comments. Whitespace is insignificant beyond separating tokens. Semicolons begin comments that extend to end of line.

```
; This is a comment (semicolon to end of line)
.equ   N   10            ; named constant
.data  arr  1 2 3 4      ; memory allocation starting at address 0

start:
    LOAD   R0, #N        ; load constant N into R0
    LOAD   R1, arr       ; R1 = base address of arr
loop:
    LOADI  R2, R1        ; R2 = mem[R1]
    SYSCALL 1            ; print R0
    ADDI   R1, R1, #1    ; R1++
    SUBI   R0, R0, #1    ; R0--
    JNZ    loop
    HALT
```

5.2 Directives

Directive	Syntax	Effect
.equ	.equ NAME value	Define a named integer constant. value can be decimal, 0x hex, 0b binary, or a char literal ‘A’. Available anywhere in the file after the directive.
.data	.data NAME v1 v2 …	Reserve consecutive memory cells. NAME resolves to the base address. Values can be decimal integers or char literals.
.org	.org ADDR	Set the .data write pointer to ADDR. Useful to place data at a specific memory location away from the default (address 0).

5.3 Operand Forms

Form	Example	Meaning

Rn	R0, R7	Register index 0–7
#decimal	#5, #-42	Signed decimal immediate
#hex	#0xFF, #0x1A	Hexadecimal immediate
#binary	#0b1010	Binary immediate
char literal	'A', ' ', '0'	ASCII / Unicode code point
label	loop, done, fact	Program address — resolved at assemble time to instruction index
.equ name	N, MAX, SIZE	Named constant — resolved at assemble time to its integer value

5.4 Labels

```
loop:                   ; bare label — binds to the next instruction
    ADD  R1, R1, R0
done: HALT              ; label and instruction on the same line (allowed)
```

Labels are case-sensitive. Forward references are resolved in the second pass. A label may appear at most once as a definition; it may be referenced any number of times.

5.5 Register Conventions

Register	Convention	Notes
R0	First argument / return value	Caller-saved. SYSCALL 1 (PRINT_INT) reads from R0. SYSCALL 4 (READ_INT) writes to R0.
R1–R6	General purpose	Caller-saved. No hardware enforcement. Use freely.
R7	Stack pointer (SP)	Initialised to 255. PUSH decrements SP; POP increments SP.

6. Instruction Set Reference

The ISA contains 46 instructions. In substrate_mode=True, all arithmetic instructions (ADD/SUB/MUL/DIV/MOD and their immediate variants, plus CMP/CMPI) are dispatched through SubstrateALU and operate natively on base-12 digit arrays. Bitwise and shift instructions read and write via cells. See Chapter 9 for the full SubstrateALU mechanics.

6.1 Arithmetic and Logic (0x01–0x0F)

Op	Mnemonic	Operands	Semantics / Flags
01	LOAD	R, #imm	Rd ← imm | Z N
02	MOV	R, R	Rd ← Rs | Z N

03	ADD	R, R, R	Rd ← Rs1 + Rs2 | Z N C V — SubstrateALU carry/borrow
04	SUB	R, R, R	Rd ← Rs1 − Rs2 | Z N C V — SubstrateALU carry/borrow
05	AND	R, R, R	Rd ← Rs1 & Rs2 (magnitudes) | Z N
06	OR	R, R, R	Rd ← Rs1 | Rs2 (magnitudes) | Z N
07	XOR	R, R, R	Rd ← Rs1 ^ Rs2 (magnitudes) | Z N
08	NOT	R, R	Rd ← ~Rs & ((1<<word_bits)−1) | Z N
09	LSL	R, R, #	Rd ← Rs << amt | Z N
0A	LSR	R, R, #	Rd ← Rs >> amt | Z N
0B	MUL	R, R, R	Rd ← Rs1 × Rs2 (signed) | Z N — SubstrateALU long-mul
0C	DIV	R, R, R	Rd ← Rs1 ÷ Rs2 (truncate toward zero) | Z N — SubstrateALU Knuth D
0D	MOD	R, R, R	Rd ← Rs1 mod Rs2 (sign follows dividend) | Z N — SubstrateALU
0E	CMP	R, R	flags ← Rs1 − Rs2; registers unchanged | Z N C V — SubstrateALU.compare()
0F	HALT	(none)	halted = True; execution stops. No flags modified.

6.2 Control Flow (0x10–0x1B)

Op	Mnemonic	Condition	Notes
10	JMP	always	Unconditional jump
11	JZ	Z = 1	Jump if zero
12	JNZ	Z = 0	Jump if not zero
13	JN	N = 1	Jump if negative
14	JNN	N = 0	Jump if non-negative (positive or zero)
15	JC	C = 1	Jump if carry or borrow
16	JE	Z = 1	Synonym for JZ; use after CMP for readability
17	JNE	Z = 0	Synonym for JNZ
18	JG	Z=0 AND N=V	Signed greater-than (after CMP or SUB)
19	JL	N ≠ V	Signed less-than
1A	JGE	N = V	Signed greater-or-equal

1B	JLE	Z=1 OR N≠V	Signed less-or-equal

6.3 Immediate-Mode Arithmetic (0x20–0x25)

Op	Mnemonic	Operands	Semantics / Flags
20	ADDI	R, R, #	Rd ← Rs + imm | Z N C V — SubstrateALU (imm loaded to temp register)
21	SUBI	R, R, #	Rd ← Rs − imm | Z N C V — SubstrateALU
22	MULI	R, R, #	Rd ← Rs × imm | Z N — SubstrateALU long-mul
23	ANDI	R, R, #	Rd ← Rs & imm (magnitude) | Z N
24	ORI	R, R, #	Rd ← Rs | imm (magnitude) | Z N
25	CMPI	R, #	flags ← Rs − imm; register unchanged | Z N C V — SubstrateALU.compare()

6.4 Memory Access (0x30–0x35)

Op	Mnemonic	Operands	Semantics
30	STORE	R, addr	mem[addr] ← Rs (compile-time constant address)
31	LOAD_MEM	R, addr	Rd ← mem[addr] | Z N
32	STOREI	R, R	mem[Rb] ← Rs (indirect: address comes from register Rb)
33	LOADI	R, R	Rd ← mem[Rs] | Z N (indirect load)
34	STOREX	R, R, #	mem[Rb + offset] ← Rs (base + signed offset)
35	LOADX	R, R, #	Rd ← mem[Rb + offset] | Z N

6.5 Stack and Subroutines (0x40–0x43)

Op	Mnemonic	Operands	Semantics
40	PUSH	R	mem[SP] ← Rs; SP ← SP − 1
41	POP	R	SP ← SP + 1; Rd ← mem[SP] | Z N
42	CALL	addr	call_stack.push(PC + 1); PC ← addr
43	RET	(none)	PC ← call_stack.pop()

6.6 System Calls and Miscellaneous (0x50–0x52)

Op	Mnemonic	Operands	Semantics
50	SYSCALL	N	Dispatch to the console device by syscall number N (see table below)
51	NOP	(none)	No operation; one CPU cycle consumed. Useful for timing or code alignment.
52	CLRS	(none)	Clear Shadow flag: S ← 0. Does not affect Z, N, C, or V. Only meaningful in substrate_mode=True; in plain mode S is always 0.

Syscall Numbers

#	Name	Action
1	PRINT_INT	Print signed integer in R0 to the console buffer, followed by nothing
2	PRINT_CHAR	Print Unicode character whose code point is in R0
3	PRINT_STR	Print zero-terminated string starting at mem[R0]; reads cells until a 0 is found
4	READ_INT	Read next integer from the input queue into R0 (queue pre-loaded via cpu.console.input_queue)
5	EXIT	Set halted = True; exit code = R0 (inspectable as info['exit_code'])
6	PRINT_NL	Print a newline character
7	DUMP_REGS	Print full CPU state (R0–R7, all 5 flags, PC) to the console buffer

7. Python API Reference

7.1 run_program — One-Shot Execution

The highest-level entry point. Assembles source, creates a CPU, runs to HALT or cycle limit, and returns (cpu, info).

```
from noisecore_vm import run_program
```

```
cpu, info = run_program(
    source,                 # str: assembly source text
    substrate_mode=False,   # True = Golay-backed ShadowRegisters + SubstrateALU
    mode='SV',              # substrate mode string: 'SM' or 'SV'
    word_bits=16,           # word width: 8, 16, 32, or 64
    memory_size=256,        # number of memory cells
    trace=False,            # record full instruction trace
    max_cycles=1_000_000,   # raise RuntimeError if exceeded
)
```

info dict keys

Key	Type	Meaning
cycles	int	Total instructions executed
halted	bool	True if HALT was reached
exit_code	int	Code from SYSCALL 5 / EXIT, else 0
output	str	Everything written to the console
flags	dict	{'Z':n,'N':n,'C':n,'V':n,'S':n} at program end
registers	list[int]	Signed values of R0–R7 at program end
elapsed_s	float	Wall-clock execution time in seconds
ips	int	Instructions per second
trace_hash	str|None	SHA-256 of instruction trace, or None if trace=False
shadow_flag	int	S flag value at program end (0 or 1)
total_carry_work	int	Cumulative base-12 carry events from SubstrateALU across all registers

7.2 CPU — Manual Control

Use the CPU class directly when you need to pre-load memory, inspect state mid-run, or attach custom devices.

```
from noisecore_vm import CPU, assemble

cpu = CPU(
    num_registers=8,
    memory_size=256,
    word_bits=16,
    mode='SV',              # 'SM' or 'SV'
    substrate_mode=True,    # True for SubstrateALU + ShadowRegisters
    trace=False,
    max_cycles=1_000_000,
)

prog = assemble(source_text)

# Pre-load any .data memory initialisation
for addr, val in prog.memory_init().items():
    cpu._write_mem(addr, val)

info = cpu.run(prog.instructions, verbose=False)
```

7.3 Useful CPU Methods

Method / Attribute	Returns	Description
cpu._read_reg(i)	int	Signed integer value of register i
cpu._write_reg(i, value)	None	Write signed value; latches S flag if shadow_active
cpu._read_mem(addr)	int	Signed value of memory[addr]
cpu._write_mem(addr, value)	None	Write signed value to memory[addr]
cpu.format_state()	str	Human-readable register + flag dump including S flag
cpu.reset()	None	Hard reset: clears all registers, memory, flags (incl. S), call_stack, trace
cpu.substrate_fingerprint()	dict	Shadow and geometric summary — see §8.6
cpu.execute((opcode,...))	None	Execute a single instruction tuple directly
cpu.console.output()	str	All text printed by the program so far
cpu.console.input_queue	list[int]	Pre-load integers for SYSCALL 4 / READ_INT
cpu.call_stack	list[int]	Current return address stack (inspectable)
cpu.cycles	int	Instructions executed so far
cpu.flags	dict	{'Z','N','C','V','S'} — current flag state
cpu.halted	bool	True after HALT or SYSCALL 5

7.4 Assembler

```
from noisecore_vm import assemble, disassemble

prog = assemble(source_text, data_origin=0)
# prog.instructions  -> list of (opcode, *operands) tuples
# prog.labels        -> dict {name: instruction_index}
# prog.constants     -> dict {name: int}  (from .equ)
# prog.data_layout   -> dict {name: (start_addr, [values])}
# prog.memory_init() -> dict {addr: value}  (all .data values)

listing = disassemble(prog.instructions, labels=prog.labels)
print(listing)   # human-readable annotated listing
```

7.5 SubstrateALU — Direct Access

SubstrateALU can be used directly for research outside the CPU context.

```
from noisecore_vm import SubstrateALU, ShadowRegister

# Low-level digit operations
da = SubstrateALU._to_digits(1836)   # -> [0, 3, 7, 10]  (little-endian
base-12)
db = SubstrateALU._to_digits(1839)   # -> [3, 3, 7, 10]
digits, carry_out, carry_events = SubstrateALU._add_digits(da, db)
```

```
result = sum(d * 12**i for i, d in enumerate(digits))
print(result)          # 3675 (1836 + 1839)
print(carry_events)    # number of digit positions that generated a carry

# Signed register arithmetic
ra = ShadowRegister(); ra.write(1836)
rb = ShadowRegister(); rb.write(1839)
rd = ShadowRegister()
result_neg = SubstrateALU.mul(ra, False, rb, False, rd)
print(rd.read())       # 3376404  (1836*1839, via base-12 long multiplication)

# Division
rn = ShadowRegister(); rn.write(3376404)
rq = ShadowRegister(); rr = ShadowRegister()
SubstrateALU.divmod(rn, False, rb, False, rq, rr)
print(rq.read())       # 1836   (quotient)
print(rr.read())       # 0      (remainder)
```

7.6 ShadowRegister

```
from noisecore_vm import ShadowRegister

r = ShadowRegister(mode='SM')
r.write(5)                     # standard write: integer -> base-12 digits ->
cells
r.write_from_alu([5, 0], 1)    # ALU write: digit list direct to cells,
                               # carry_events added to carry_work

r.shadow_active                # True if any cell digit > 4 (Phenomenal limit)
r.carry_work                   # cumulative base-12 carry events
r.shadow_fingerprint()         # {'shadow_active', 'shadow_cell_count',
                               #  'shadow_cell_indices', 'shadow_digits',
                               #  'carry_work', 'phenomenal_limit', 'law'}
```

7.7 Custom Devices

```
from noisecore_vm import CPU, Device

class TimerDevice(Device):
    name = 'timer'
    def on_syscall(self, cpu, num):
        if num == 10:            # SYSCALL 10 -> read cycle count into R0
            cpu._write_reg(0, cpu.cycles)
        elif num == 11:          # SYSCALL 11 -> read shadow flag into R0
            cpu._write_reg(0, cpu.flags['S'])
        else:
            raise NotImplementedError(f'Unknown syscall {num}')
    def reset(self): pass

cpu = CPU(substrate_mode=True)
cpu.devices.append(TimerDevice())
```

8. Substrate Mode and the Golay Geometry

8.1 What the Substrate Is

Each ShadowRegister is a base-12 positional accumulator. Writing the value 144 decomposes it into base-12 digits [0, 0, 1] ($144 = 1 \times 12^2$) and assigns each digit to a separate NoiseCellV3. Each cell holds a 24-bit substrate vector selected from SubstrateLibrary.PERFECT_V1 — a specific Golay [24,12,8] codeword with Hamming weight 9.

When digit k is written into a cell, the Golay engine measures the syndrome weight of the codeword after k positional perturbations. The difference between the baseline syndrome weight (7 for PERFECT_V1) and the perturbed syndrome weight is the displacement — a geometric quantity measuring how far the digit has moved the manifold point.

8.2 The Elastic Limit and the Shadow Regime

PERFECT_V1's displacement curve is linear for digits 0 through 4 — the Phenomenal (elastic) regime. For digits 5 through 11 the curve is non-linear: the manifold vector has crossed the Voronoi boundary of its local packing sphere and snapped to a new geometric anchor.

Digit	Displacement	sm_consistent	Regime
0	0	True	Phenomenal (linear)
1	1	True	Phenomenal (linear)
2	2	True	Phenomenal (linear)
3	3	True	Phenomenal (linear)
4	4	True	Phenomenal ← elastic limit
5	1	False	Shadow Regime begins
6	4	False	Shadow Regime
7–11	varies	False	Shadow Regime (Noumenal fold)

sm_consistent = False does not mean the value is wrong. The logical value is always recoverable via register.read(). It means the substrate has folded into a different geometric anchor — the computation is in the Shadow Regime.

8.3 The Shadow Processor (LAW_SHADOW_PROCESSOR_001)

The Shadow Processor is the geometric monitoring layer that detects when computation crosses the Voronoi boundary. The excess geometric frustration folds into the 12-bit Noumenal (hidden) space of the Golay code — hence "Shadow".

ShadowRegister extends NoiseRegisterV3 with two observable quantities:

- shadow_active : bool — True when any cell digit > 4. Updated on every write() or write_from_alu().
- carry_work : int — Cumulative base-12 carry events across all writes (LAW_CARRY_WORK_001).

```
from noisecore_vm import ShadowRegister

r = ShadowRegister(mode='SM')
r.write(3)                    # digit 3 — Phenomenal
print(r.shadow_active)        # False

r.write(5)                    # digit 5 — Shadow
print(r.shadow_active)        # True
print(r.shadow_fingerprint())
# {'shadow_active': True,
#  'shadow_cell_count': 1,
#  'shadow_cell_indices': [0],
```

```
#  'shadow_digits': [5],
#  'carry_work': 5,
#  'phenomenal_limit': 4,
#  'law': 'LAW_SHADOW_PROCESSOR_001 / LAW_CARRY_WORK_001'}
```

8.4 Carry Work (LAW_CARRY_WORK_001)

ShadowRegister.carry_work counts the cumulative number of base-4 carry events across all writes. A base-4 carry occurs when (a_digit + b_digit) ≥ 4 at any base-4 digit position when adding two values. This is the geometric binding work — the frustration generated during value assembly in the digit manifold.

Classical binding energy (e.g. nuclear binding energy per nucleon) is the macroscopic analogue of this quantity. The count_base4_carries() function computes this directly:

```
from noisecore_vm import count_base4_carries

# Proton (1836) + Neutron (1839) assembly work
work = count_base4_carries(1836, 1839)
print(work)   # base-4 carry events

# CPU-level aggregate after a program run:
cpu, info = run_program(src, substrate_mode=True)
print(info['total_carry_work'])   # summed across all registers
```

8.5 The S (Shadow) Flag and CLRS

The S flag is a latching flag in cpu.flags['S']. It is set when _write_reg() triggers shadow_active on any ShadowRegister. Once set, S remains 1 until:

- CLRS instruction (0x52) — clears S without affecting Z/N/C/V
- cpu.reset() — full hard reset

In substrate_mode=False, S is always 0 (no ShadowRegisters, so no shadow detection).

Using CLRS for Section-Level Shadow Probing

```
; Test whether a block of computation triggers Shadow Regime
    CLRS                ; clear S from any previous state

; --- section under test ---
    LOAD  R0, #3
    LOAD  R1, #3
    ADD   R2, R0, R1     ; result=6, digit=6 > 4 -> S latches to 1
    SYSCALL 7            ; DUMP_REGS shows S=1

    CLRS                 ; reset for next block
    LOAD  R0, #2
    ADD   R2, R2, R0     ; within phenomenal range - S stays 0
    HALT

; In Python after run():
; info['shadow_flag'] == 1  -> section entered Shadow Regime
; info['total_carry_work']  -> binding work performed
```

8.6 Fingerprints and Geometric Observables

NRCI — Noise-Resonance-Coherence Index

Computed as 10 / (10 + |hw − 12|) where hw is the Hamming weight of the substrate vector. For PERFECT_V1 (hw = 9), NRCI is always 0.7692. This is a property of the substrate geometry, not of the stored values — it would vary only if different substrate types were mixed across registers.

magnitude_sha

SHA-256 (first 16 hex chars) of the magnitude pattern across all registers and the first 32 memory cells. This IS computation-dependent — it changes every time registers or memory are written, giving a deterministic geometric signature of the CPU's current numeric state.

substrate_fingerprint() Return Values

```
fp = cpu.substrate_fingerprint()
fp['mode']              # 'substrate' or 'plain'
fp['nrci_mean']         # 0.7692 for PERFECT_V1 (substrate constant)
fp['magnitude_sha']     # changes with computation
fp['shadow_flag']       # mirrors cpu.flags['S']
fp['shadow_registers']  # list of register indices currently in Shadow Regime
fp['total_carry_work']  # cumulative base-12 carry events (all registers)
```

8.7 Trace Hash for Reproducibility

When trace=True, the CPU records a TraceEntry for every instruction: (pc, instr, mnemonic, flags_before, flags_after, cycle). The same program always produces the same SHA-256 hash — verified for all 8 example programs.

```
cpu, info = run_program(src, trace=True)
print(info['trace_hash'])   # e.g. '5ca7e1742f5e0ed...'

cpu2, info2 = run_program(src, trace=True)
assert info['trace_hash'] == info2['trace_hash']   # always True
```

8.8 Research Script Pattern

```
from noisecore_vm import run_program
import json

src = open('examples/05_prime_sieve.nca').read()
cpu, info = run_program(src, substrate_mode=True, trace=True)
fp = cpu.substrate_fingerprint()

record = {
    'program':          '05_prime_sieve.nca',
    'output':           cpu.console.output().strip(),
    'cycles':           info['cycles'],
    'trace_hash':       info['trace_hash'],
    'magnitude_sha':    fp['magnitude_sha'],
    'nrci_mean':        fp['nrci_mean'],
    'shadow_flag':      fp['shadow_flag'],
    'total_carry_work': fp['total_carry_work'],
}
print(json.dumps(record, indent=2))
```

9. SubstrateALU — Native Base-12 Arithmetic

This chapter describes the SubstrateALU class introduced in v1.2.0. All CPU arithmetic in substrate_mode=True now operates digit-by-digit on base-12 cell arrays. Python arithmetic operators are never applied to full register values.

9.1 Design Rationale

A base-12 digit-chain ALU is the correct implementation for a Golay-backed register machine. The Golay code's packing radius t = 3 sets the elastic limit at digit value 4. Operations within the phenomenal regime (all digits ≤4) are linear — displacement is proportional to digit value. Operations involving larger digits cross Voronoi boundaries and enter the Shadow Regime.

Base 12 is chosen because the linear (phenomenal) regime holds for digit values 0–4, and 12 is the base of the positional accumulator. A carry in base 12 corresponds to the cell reaching the boundary of its local packing sphere.

The only Python arithmetic performed on intermediate values is single-digit operations bounded to the following ranges:

Law	Operation	Max intermediate value	How bounded
LAW_SUBSTRATE_ALU_001	ADD per digit: a + b + carry	23	11 + 11 + 1 = 23
LAW_SUBSTRATE_ALU_001	SUB per digit: a − b − borrow	−11 … 11	corrected by adding 12 if negative
LAW_SUBSTRATE_ALU_002	MUL inner: acc + a_i×b_j + carry	143	11 + 11×11 + 11 = 143
LAW_SUBSTRATE_ALU_003	DIV trial: 2-digit window / lead digit	143	(11×12 + 11) / 1 = 143
LAW_SUBSTRATE_ALU_004	Bitwise: cell read → binary op	N/A	cell-read then write back via digit path

9.2 Addition and Subtraction (LAW_SUBSTRATE_ALU_001)

ADD and SUB are performed digit-by-digit with carry or borrow propagation. Each digit step computes s = a_digit + b_digit + carry, where s ≤ 23. If s ≥ 12, carry = 1 and digit = s − 12; otherwise carry = 0.

```
# 1836 + 1839 worked through in base-12:
# 1836 = [0, 3, 7, 10]  (digit0 ... digit3, little-endian)
# 1839 = [3, 3, 7, 10]
#
# digit0:  0+3+0 =  3  -> store 3,  carry=0   (no carry)
# digit1:  3+3+0 =  6  -> store 6,  carry=0   (no carry)
# digit2:  7+7+0 = 14  -> store 2,  carry=1   (CARRY EVENT)
# digit3: 10+10+1= 21  -> store 9,  carry=1   (CARRY EVENT)
# digit4:  0+0+1 =  1  -> store 1,  carry=0
#
# Result: [3,6,2,9,1] = 3*1 + 6*12 + 2*144 + 9*1728 + 1*20736 = 3675
# carry_events = 2

from noisecore_vm import SubstrateALU
da = SubstrateALU._to_digits(1836)
```

```
db = SubstrateALU._to_digits(1839)
digits, carry_out, ce = SubstrateALU._add_digits(da, db)
print(sum(d*12**i for i,d in enumerate(digits)))  # 3675
print(ce)                                          # 2
```

9.3 Multiplication (LAW_SUBSTRATE_ALU_002)

Base-12 long multiplication. For each pair of digits (a_i, b_j), the inner product is a_i × b_j ≤ 121. Adding the running accumulator digit (≤11) and the carry (≤11) gives a maximum intermediate value of 143.

```
# 1836 * 1839 via base-12 long multiplication
da = SubstrateALU._to_digits(1836)
db = SubstrateALU._to_digits(1839)
digits = SubstrateALU._mul_digits(da, db)
print(sum(d*12**i for i,d in enumerate(digits)))  # 3376404
```

9.4 Division and Modulo (LAW_SUBSTRATE_ALU_003)

Base-12 long division using Knuth Algorithm D adapted for base 12. The trial-quotient step uses a 2-digit window of the running remainder (≤143) divided by the leading divisor digit. An adjustment loop corrects over-estimation and runs at most 2 iterations per digit position.

```
# 3376404 / 1839 via Knuth Algorithm D (base-12)
da = SubstrateALU._to_digits(3376404)
db = SubstrateALU._to_digits(1839)
q_d, r_d = SubstrateALU._divmod_digits(da, db)
q = sum(d*12**i for i,d in enumerate(q_d))
r = sum(d*12**i for i,d in enumerate(r_d))
print(q, r)   # 1836 0
```

9.5 Bitwise Operations and Shifts (LAW_SUBSTRATE_ALU_004)

AND, OR, XOR, NOT, LSL, and LSR read the unsigned magnitude from cells, apply the binary operation, then write the result back via the write_from_alu() digit path. This is the maximum UBP-native form achievable for binary operations on a base-12 substrate: the store/retrieve path is always through the cells, preserving the geometric encoding.

Binary operations reconstruct the integer from cell digits before applying the bitwise operator, because binary operations have no natural base-12 analogue. The critical property is preserved: the result is always written back as a digit array via write_from_alu(), so the substrate encodes and fingerprints the result geometrically.

9.6 Signed Arithmetic — Sign Convention

SubstrateALU operates exclusively on unsigned magnitudes. Sign is resolved at the CPU dispatch layer using cpu._signs[]. The four sign cases for addition are:

sign_a	sign_b	Resolves to	Method
False	False	(+a) + (+b) = +(a+b)	_add_digits(da, db)
True	True	(−a) + (−b) = −(a+b)	_add_digits(da, db), negate result
False	True	(+a) + (−b) = a−b or −(b−a)	_sub_digits(larger, smaller)

True	False	(−a) + (+b) = b−a or −(a−b)	_sub_digits(larger, smaller)

SUB is implemented as ADD with the second operand's sign flipped. MUL and DIV resolve sign by XOR of input signs and operate on magnitudes only.

9.7 write_from_alu()

After the ALU produces a digit list, it calls ShadowRegister.write_from_alu(digits, carry_events) to write results directly into cells without re-decomposing an integer. This method:

- Grows the cell array if the digit list is longer than current capacity
- Writes each digit directly to its NoiseCellV3
- Zeroes any excess cells beyond the new digit count
- Reconstructs _value from the digit array (for read() compatibility)
- Adds carry_events to carry_work (LAW_CARRY_WORK_001)
- Updates shadow_active (any digit > 4 → True)

9.8 Immediate Operations

ADDI, SUBI, MULI, ANDI, ORI, CMPI load the immediate value into a temporary ShadowRegister and then delegate to the signed ALU methods (add, sub, mul, bitwise_and, bitwise_or, compare). This means even immediate-mode arithmetic is performed natively on digit arrays.

9.9 Performance

Base-12 long arithmetic is slower than Python integer arithmetic. Typical overhead versus plain mode:

Operation	Overhead vs plain mode	Reason
ADD / SUB	10–15×	Digit-by-digit carry/borrow loop
MUL	20–50×	O(m×n) digit-pair iterations
DIV / MOD	30–80×	Knuth D trial-quotient loop per digit
Bitwise / shifts	8–12×	Cell read + binary op + digit write

This overhead is a deliberate research trade-off. For throughput-sensitive work, use substrate_mode=False — results are identical.

10. Command-Line Interface

10.1 run — Assemble and Execute

```
noisecore run FILE.nca [options]
```

```
--substrate       Enable Golay-backed ShadowRegisters + SubstrateALU
--word-bits N     Word width: 8, 16 (default), 32, or 64
--memory N        Memory size in cells (default 256)
--max-cycles N    Cycle limit before RuntimeError (default 1,000,000)
--trace           Record and hash the full instruction trace
--verbose, -v     Print each instruction as it executes
--stats           Print cycle count, IPS, substrate fingerprint to stderr
```

```
$ noisecore run examples/07_gcd.nca --substrate --stats
18
[stats] cycles=33  halted=True  exit=0  elapsed=0.0019s
[stats] substrate NRCI mean = 0.7692
[stats] magnitude SHA-16   = a3f9c12e00b4d871
[stats] shadow_flag        = 0
[stats] total_carry_work   = 12
```

Verbose output shows every instruction, its operands, and the full flag state including S:

```
$ noisecore run examples/02_factorial_iter.nca --substrate --verbose
PC=0000  LOAD   (0, 5)    flags={'Z':0,'N':0,'C':0,'V':0,'S':0}
PC=0001  LOAD   (1, 1)    flags={'Z':0,'N':0,'C':0,'V':0,'S':1}
PC=0002  CMP    (0, 1)    flags={'Z':0,'N':0,'C':0,'V':0,'S':1}
...
```

10.2 asm, disasm, info, repl

Command	Usage
noisecore asm FILE.nca	Print human-readable listing: addresses, labels, bytecode, mnemonics
noisecore disasm FILE.nca	Assemble then disassemble — verifies round-trip fidelity
noisecore info	Print full ISA table (all 46 opcodes, operand forms, flags)
noisecore repl [--substrate]	Interactive single-instruction REPL. Branch instructions are not supported in the REPL context.

11. Practical Application Patterns

11.1 Counted Loop

```
    LOAD  R0, #10        ; counter
loop:
    CMPI  R0, 0
    JLE   done
    ; ... loop body ...
    SUBI  R0, R0, #1
    JMP   loop
done:
    HALT
```

11.2 Array Iteration via Pointer

```
.data arr  10 20 30 40 50
```

```
.equ  N    5
    LOAD  R5, arr        ; R5 = base address of arr
    LOAD  R6, #N         ; R6 = remaining count
loop:
    CMPI  R6, 0
    JLE   done
    LOADI R0, R5         ; R0 = mem[R5]
    SYSCALL 1            ; print R0
    SYSCALL 6            ; newline
    ADDI  R5, R5, #1     ; advance pointer
    SUBI  R6, R6, #1     ; decrement count
    JMP   loop
done:
    HALT
```

11.3 Recursive Subroutine — Factorial

```
; Factorial: R0 = n on entry, R0 = n! on exit
fact:
    CMPI    R0, 1
    JLE     fact_base
    PUSH    R0           ; save n
    SUBI    R0, R0, #1   ; n-1
    CALL    fact         ; R0 = (n-1)!
    POP     R1           ; restore n
    MUL     R0, R0, R1   ; n * (n-1)!  [SubstrateALU in substrate mode]
    RET
fact_base:
    LOAD    R0, #1
    RET

    LOAD    R0, #10
    CALL    fact
    SYSCALL 1            ; print 10!
    SYSCALL 6
    HALT
```

11.4 Memory-Mapped String Output

```
.data msg  'H' 'e' 'l' 'l' 'o' ',' ' ' 'W' 'o' 'r' 'l' 'd' 0
    LOAD    R0, msg
    SYSCALL 3            ; PRINT_STR (reads until zero terminator)
    SYSCALL 6
    HALT
```

11.5 Shadow Probe Pattern

Use CLRS to bracket sections and inspect info['shadow_flag'] after run to detect whether a computation crossed the elastic limit:

```
    CLRS                 ; clear sentinel before section

; === section under test ===
    LOAD  R0, #5         ; digit 5 -> Shadow Regime
    LOAD  R1, #3
    ADD   R2, R0, R1
; === end of section ===

    SYSCALL 7            ; DUMP_REGS shows S=1
    HALT
```

```
; Python:
; cpu, info = run_program(src, substrate_mode=True)
; if info['shadow_flag']:  print('section entered Shadow Regime')
; print('carry work:', info['total_carry_work'])
```

11.6 Binding Work Research Pattern

Measure the geometric binding work (base-4 carry events) of nucleon assembly directly using SubstrateALU:

```
from noisecore_vm import SubstrateALU

proton  = 1836
neutron = 1839

# Deuterium (D-2): proton + neutron
da = SubstrateALU._to_digits(proton)
db = SubstrateALU._to_digits(neutron)
_, _, ce = SubstrateALU._add_digits(da, db)
print(f'D-2 binding work: {ce} base-12 carry events')

# Helium-4: two protons + two neutrons
dp = SubstrateALU._to_digits(2 * proton)
dn = SubstrateALU._to_digits(2 * neutron)
_, _, ce4 = SubstrateALU._add_digits(dp, dn)
print(f'He-4 binding work: {ce4} base-12 carry events')
```

11.7 Full Research Record

```
from noisecore_vm import run_program
import json, datetime

src = open('examples/05_prime_sieve.nca').read()
cpu, info = run_program(src, substrate_mode=True, trace=True)
fp = cpu.substrate_fingerprint()

record = {
    'timestamp':        datetime.datetime.utcnow().isoformat(),
    'program':          '05_prime_sieve.nca',
    'output':           cpu.console.output().strip(),
    'cycles':           info['cycles'],
    'elapsed_s':        info['elapsed_s'],
    'trace_hash':       info['trace_hash'],
    'magnitude_sha':    fp['magnitude_sha'],
    'nrci_mean':        fp['nrci_mean'],
    'shadow_flag':      fp['shadow_flag'],
    'shadow_registers': fp['shadow_registers'],
    'total_carry_work': fp['total_carry_work'],
}
with open('research_log.jsonl', 'a') as f:
    f.write(json.dumps(record) + '\n')
```

12. Debugging

12.1 Verbose Execution

The --verbose flag (or verbose=True in run_program) prints every instruction as it executes, including flag state:

```
$ noisecore run myprog.nca --verbose
PC=0000  LOAD      (0, 5)   flags={'Z':0,'N':0,'C':0,'V':0,'S':0}
PC=0001  LOAD      (1, 7)   flags={'Z':0,'N':0,'C':0,'V':0,'S':1}
PC=0002  ADD       (2,0,1)  flags={'Z':0,'N':0,'C':0,'V':0,'S':1}
...
```

The S flag appears in verbose output and may change on any instruction that writes a register in substrate mode.

12.2 DUMP_REGS Syscall

```
LOAD    R0, #42
LOAD    R1, #100
SYSCALL 7            ; prints R0..R7, all 5 flags, PC
HALT
```

12.3 Trace Hash for Reproducibility

The trace hash is a SHA-256 fingerprint of the complete instruction trace — every PC value, instruction, and flag state. The same program always produces the same hash.

```
cpu, info = run_program(src, trace=True)
print(info['trace_hash'])   # stable across runs, versions, and modes

# Compare substrate and plain mode:
cpu_s, info_s = run_program(src, substrate_mode=True,  trace=True)
cpu_p, info_p = run_program(src, substrate_mode=False, trace=True)
# Hashes should be identical if computation is correct
```

12.4 Inspecting Shadow State

```
cpu, info = run_program(src, substrate_mode=True)

# Top-level summary:
print(info['shadow_flag'])        # 1 if any register hit Shadow during run
print(info['total_carry_work'])   # total base-12 carry events

# Per-register detail via fingerprint:
fp = cpu.substrate_fingerprint()
print(fp['shadow_registers'])     # e.g. [0, 2] — R0 and R2 in Shadow Regime

# Register-level breakdown:
for i, reg in enumerate(cpu.registers):
    if hasattr(reg, 'shadow_fingerprint'):
        sf = reg.shadow_fingerprint()
        if sf['shadow_active']:
            print(f'R{i}: shadow digits={sf["shadow_digits"]}
carry_work={sf["carry_work"]}')
```

12.5 Inspecting SubstrateALU Operations

```
from noisecore_vm import SubstrateALU
```

```
# Check digit decomposition
print(SubstrateALU._to_digits(1836))  # [0, 3, 7, 10]

# Check a specific addition
da = SubstrateALU._to_digits(5)
db = SubstrateALU._to_digits(8)
digits, carry_out, ce = SubstrateALU._add_digits(da, db)
result = sum(d*12**i for i,d in enumerate(digits))
print(f'5+8={result}  carry_events={ce}  carry_out={carry_out}')
# 5+8=13, digit-level: 5+8=13>=12 -> store 1, carry=1
# carry_events=1 (one digit position generated a carry)
```

12.6 Common Errors and Causes

Error	Cause	Fix
RuntimeError: CPU exceeded max_cycles	Infinite loop (missing HALT or broken loop condition)	Add --max-cycles N or fix loop exit
ZeroDivisionError	DIV or MOD with Rs2 = 0	Guard with CMPI Rs2, 0 / JE error_handler
IndexError: Memory out of bounds	STORE/STOREI/STOREX with out-of-range address	Check pointer arithmetic; increase memory_size
OverflowError: Stack overflow on PUSH	SP went below 0	Reduce stack depth or increase memory_size
RuntimeError: RET with empty call stack	Unmatched RET	Every RET must be reached via CALL
AssemblerError: Unknown mnemonic	Typo in instruction name	Run noisecore info to see all 46 opcodes

13. Known Limitations

13.1 No Floating-Point

The ISA is integer-only. Use fixed-point representation for fractional values (e.g. work in milliradians or per-mil rather than floating-point). The Golay substrate and SubstrateALU are also integer-only — NRCI and carry_work are computed from Hamming weights and digit counts respectively.

13.2 No Interrupts or Exceptions

The CPU has no interrupt vector. Divide-by-zero, stack overflow, and memory bounds violations raise Python exceptions in the calling code. Guard all division operations with an explicit zero-check before the instruction.

13.3 Bitwise Operations on Magnitudes Only

AND, OR, XOR, NOT, LSL, and LSR operate on the unsigned magnitude stored in the register. If you need bitwise manipulation of a signed value, extract the sign manually, operate on the

magnitude, and restore the sign via LOAD of the result with the correct sign.

13.4 8 Registers, 256 Memory Cells

These defaults are configurable (CPU(num_registers=16, memory_size=1024)). The SP convention remains at register index 7 regardless of num_registers. Larger configurations increase SubstrateALU overhead proportionally.

13.5 REPL Does Not Support Branch Instructions

Branch instructions (JMP, JZ, JG, JLE, etc.) require a full program context to resolve label addresses. Any branch instruction typed in the REPL will produce a warning and be skipped. Use noisecore run for programs with control flow.

13.6 Shadow Detection is Observational

The S flag and shadow_active tell you that a value has entered the Shadow Regime. They do not quantify how far into the Shadow Regime a computation has gone. For depth-of-shadow analysis, inspect the displacement values of individual cells via substrate_verify() on each NoiseCellV3.

13.7 SubstrateALU Performance

Base-12 digit-chain arithmetic is 10–80× slower than Python integer arithmetic depending on value size and operation. This is a deliberate research trade-off, not a bug. Use substrate_mode=False for throughput-sensitive work — all computed results are identical.

Appendix A: Instruction Quick Reference

Mnemonic	Op	Form	Flags	Substrate ALU?
LOAD	01	R, #imm	Z N	no
MOV	02	R, R	Z N	no
ADD	03	R, R, R	Z N C V	carry/borrow
SUB	04	R, R, R	Z N C V	carry/borrow
AND	05	R, R, R	Z N	LAW_004
OR	06	R, R, R	Z N	LAW_004
XOR	07	R, R, R	Z N	LAW_004
NOT	08	R, R	Z N	LAW_004
LSL	09	R, R, #	Z N	LAW_004
LSR	0A	R, R, #	Z N	LAW_004
MUL	0B	R, R, R	Z N	long-mul
DIV	0C	R, R, R	Z N	Knuth D
MOD	0D	R, R, R	Z N	Knuth D
CMP	0E	R, R	Z N C V	compare()

HALT	0F	(none)	--	no
JMP	10	addr	--	no
JZ	11	addr	--	no
JNZ	12	addr	--	no
JN	13	addr	--	no
JNN	14	addr	--	no
JC	15	addr	--	no
JE	16	addr	--	no
JNE	17	addr	--	no
JG	18	addr	--	no
JL	19	addr	--	no
JGE	1A	addr	--	no
JLE	1B	addr	--	no
ADDI	20	R, R, #	Z N C V	carry/borrow
SUBI	21	R, R, #	Z N C V	carry/borrow
MULI	22	R, R, #	Z N	long-mul
ANDI	23	R, R, #	Z N	LAW_004
ORI	24	R, R, #	Z N	LAW_004
CMPI	25	R, #	Z N C V	compare()
STORE	30	R, addr	--	no
LOAD_MEM	31	R, addr	Z N	no
STOREI	32	R, R	--	no
LOADI	33	R, R	Z N	no
STOREX	34	R, R, #	--	no
LOADX	35	R, R, #	Z N	no
PUSH	40	R	--	no
POP	41	R	Z N	no
CALL	42	addr	--	no
RET	43	(none)	--	no
SYSCALL	50	N	--	no
NOP	51	(none)	--	no
CLRS	52	(none)	clears S	no

Green rows in the Substrate ALU column indicate digit-chain operations. LAW_004 = cell-read → binary op → digit-write.

Appendix B: Example Programs

File	Demonstrates	Cycles (plain)
01_hello.nca	PRINT_CHAR syscall, char literals	12
02_factorial_iter.nca	Iterative loop, CMPI/JG, MUL (SubstrateALU in substrate mode)	38
03_factorial_recursive.nca	CALL/RET, PUSH/POP, recursion, MUL (SubstrateALU)	49
04_bubble_sort.nca	LOADI/STOREI, indexed memory, .data arrays	450
05_prime_sieve.nca	Sieve of Eratosthenes, MOD (SubstrateALU Knuth D)	908

06_fibonacci.nca	PUSH/POP for caller-save, ADD chain (SubstrateALU)	167
07_gcd.nca	Euclidean GCD via subroutine, MOD (SubstrateALU)	33
08_string_reverse.nca	String traversal, reverse copy, PRINT_STR	172

All 8 programs produce identical output in substrate_mode=True and substrate_mode=False. The SubstrateALU performs the arithmetic by a different path but always reaches the same result.

Appendix C: Version History

v1.0.0 — Initial Release

First public release of NoiseCore VM. Established the complete ISA (45 instructions), two-pass assembler, disassembler, CLI, 8 example programs, substrate mode with NoiseRegisterV3 / NoiseCellV3 / GolayCodeEngine, NRCI and magnitude SHA fingerprints.

v1.1.0 — Shadow Processor

Area	Change
ShadowRegister	New class replacing NoiseRegisterV3 for CPU registers in substrate_mode=True. Adds shadow_active, carry_work, shadow_fingerprint().
count_base4_carries()	New module-level function. Counts base-4 carry events. LAW_CARRY_WORK_001.
S (Shadow) flag	Fifth CPU flag. Latching — set by any _write_reg() that triggers shadow_active. Cleared by CLRS or reset().
CLRS (0x52)	New opcode. Clears S flag without affecting Z/N/C/V. Opcode count: 45 → 46.
cpu.substrate_fingerprint()	Three new fields: shadow_flag, shadow_registers, total_carry_work.
cpu.run() return dict	Two new fields: shadow_flag, total_carry_work.
test_shadow.py	33 new tests covering count_base4_carries, ShadowRegister, CPU S flag, CLRS. Total: 151/151.
docs/ARCHITECTURE.md	New Shadow Processor section with displacement table, S flag semantics, CLRS description.

v1.2.0 — SubstrateALU (Native Base-12 Arithmetic)

Area	Change
SubstrateALU (new class)	Full digit-chain ALU: _add_digits, _sub_digits, _mul_digits, _divmod_digits, compare, add, sub, mul, divmod, bitwise ops, shifts, _to_digits.

LAW_SUBSTRATE_ALU_001	ADD/SUB via base-12 carry/borrow propagation; intermediate values ≤23.
LAW_SUBSTRATE_ALU_002	MUL via base-12 long multiplication; single-digit products ≤121; inner accumulation ≤143.
LAW_SUBSTRATE_ALU_003	DIV/MOD via Knuth Algorithm D adapted for base 12; trial-quotient values ≤143; at most 2 adjustment steps per digit position.
LAW_SUBSTRATE_ALU_004	AND/OR/XOR/NOT/LSL/LSR: cell-read → binary op → digit-write (write_from_alu).
ShadowRegister.write_from_alu()	New method. Digit list → cells directly; updates carry_work and shadow_active in one step.
CPU arithmetic ops	ADD/SUB/MUL/DIV/MOD/ADDI/SUBI/MULI/ANDI/ORI/CMP/CMPI dispatch through SubstrateALU in substrate_mode=True. Plain mode unchanged.
_add_flags() / _sub_flags()	Accept explicit carry_out/borrow_out from ALU (no recomputation from integer result).
test_substrate_alu.py	57 new tests: digit ops, signed ALU, CPU integration, cross-mode correctness. Total: 208/208.
pyproject.toml / __init__.py	Version 1.2.0; SubstrateALU and write_from_alu exported publicly.
Previous limitation resolved	v1.1.0 §12.7 (“Substrate Computation is Verified, Not Mediated”) is superseded. Arithmetic in substrate_mode=True is now performed natively by the SubstrateALU digit-chain.